National University of Computer and Emerging Sciences

Artificial Intelligence (AL-2002)

Lab Mid Exam

Total Time (hr): 1.5
Total Points: 50
Total Questions: 03

Paper B

Date: March 18th, 2025

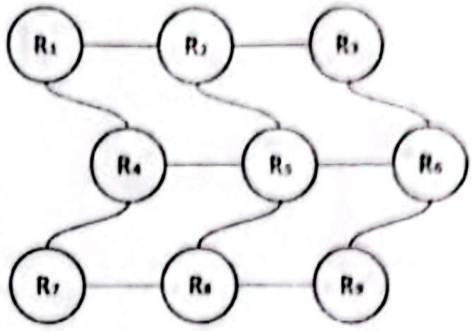
Course Instructor(s)

Mr. Talha Shahid, Miss. Alishba Subhani

Student Name _____ Roll No _____ Section _____ Student Signature _____

Question # 01 (LLO: 1) **[18 Points, 8 Weightage]**

In a **warehouse automation system**, robots are tasked with retrieving packages from storage racks. The storage racks are represented as an unweighted graph where each rack is a node, and accessible paths between racks are edges. A robot perceives its environment by detecting nearby racks and available paths. The robot's goal is to find the shortest path to a specified rack using **Iterative Deepening Depth-First Search (IDDFS)**.



Robot's Starting Rack: **R1** | Robot's Goal: **R8**

Tasks:

1. Define the architecture of a goal-based robot agent and an environment class representing the warehouse layout. **[07]**
2. Implement the **Iterative Deepening Depth-First Search (IDDFS)** function within the agent class to find the shortest path to the goal rack. **[08]**
3. Create a `run_agent()` function to simulate the robot's behavior in the warehouse and display the shortest path found. **[03]**

Question # 02 (LLO: 3) **[18 Points, 9 Weightage]**

In a **treasure hunting game**, a player must navigate an island to find a hidden treasure. The island is represented as a grid where:

- . represents a traversable path.
- # represents an impassable obstacle.
- **S** represents the starting point (player's position).
- **T** represents the treasure's location.

The player can move up, down, left, or right. The player should use **Best-First Search (BFS)** with the **Euclidean distance** to the treasure as a heuristic to find the shortest path.

National University of Computer and Emerging Sciences

S	.	.	#	.	T
.	#	.	#	.	#
.	.	.	.	#	.
#	#	.	#	.	.
.	.	.	.	.	#

S = Starting Point
T = Treasure's Location
. = Traversable Path
# = Obstacle

Task:

1. Define the heuristic function which calculates the Euclidean distance between a point and the treasure. **[03]**
2. Define a **`generate_neighbours`** function to provide all possible neighbours of a given state based on the player's allowed movements. **[04]**
3. Implement the Best-First Search (BFS) function to find the shortest path to the treasure using the Euclidean distance heuristic. **[08]**
4. Create a **`main()`** function to execute the Best-First Search function, displaying the path taken and the total number of steps required. **[03]**

Question # 03 (LLO: 4) **[14 Points, 7 Weightage]**

Genetic Algorithm for the 0/1 Knapsack Problem | refer to the provided guide.

A logistics company is optimizing the space utilization of its delivery truck by selecting the best combination of packages to carry. Each package has a **weight** and **value**, and the truck has a **maximum weight limit.**

To solve this problem, you will implement a **Genetic Algorithm (GA)**, but with **modified selection, crossover, and mutation strategies**. Some parts of the algorithm are provided, and your task is to complete the missing functions.

You **must implement** the following functions to complete the GA:

1. **Generating Random Individuals** | `def create_random_individual(num_items):`
   - Create a random binary list where each gene represents whether an item is selected (1) or not (0).
2. **Initializing the Population** | `def initialize_population(pop_size, num_items):`
   - Generate an initial set of candidate solutions using the function from Step 1.
3. **Selecting Parents** | `def tournament_selection(population, fitness_scores):`
   - Instead of picking the top 50% directly, **implement tournament selection** where:
     - Randomly select **three individuals** from the population.
     - The individual with the **highest fitness** is chosen as a parent.
4. **Performing Two-Point Crossover** | `def crossover(parent1, parent2):`
   - Implement **two-point crossover**, where:
     - Select **two random crossover points**.
     - Swap the segment between them.